

The VP-pro family: four reasons the value-profile gradient gets through

Backdrop. *VP-pro* collects the roadblocks where a fuzzer that *scores how byte-close each comparison came* (value_profile’s Hamming-similarity / CMP_MAP feedback) gets through, while a fuzzer without that gradient stays stuck. Where I2S needs the right value to be lying around so it can *paste* it, VP needs no such luck: each comparison emits a *partial-match* reward that turns a flat “wrong/right” cliff into a hill the fuzzer can climb one byte at a time. Picture a combination lock that clicks louder as each wheel nears its number — VP hears the clicks and turns toward them; plain havoc (and even exact-paste I2S, when the value is *not* already in the corpus) has no slope to follow. All the categories below share that gradient — they differ in *what* the climb buys. Counts are validated roadblocks (n); a branch joins VP-pro by its *deciding pair* (value_profile resolves, its non-VP partner blocks), not by the raw 4-char shape.

1. Operand byte-gradient climb `vp_gradient_value_distance_closure` ($n = 33$)

The dominant case. The gate is an exact multi-byte compare (a FourCC tag, a keyword, a magic constant). The non-VP arm’s best seed gets *some* of the bytes right and stalls there — it has no signal telling it the remaining bytes are wrong-but-close. The value-profile gradient scores that near-match, so VP keeps the partial hit and climbs the last bytes home. *It is not getting to the door; it is hearing the lock click toward open.*

```
// lcms cmspcs.c : cmsChannelsOfColorSpace()
// dispatch on the 4-byte ICC colour-space signature
switch (ColorSpace) {
case cmsSigMCH2Data: // <-- blocker (lcms/2288): sig ==
    'MCH2'
case cmsSig2colorData: return 2; // needs all four exact bytes
}
```

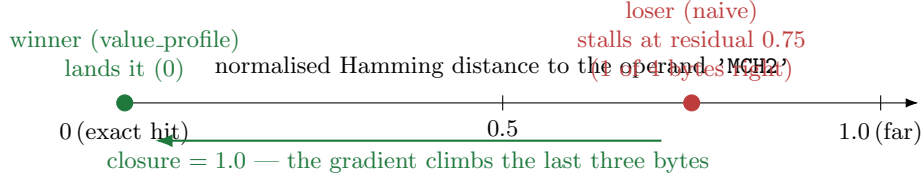
How we know it is really the gradient. We measure how *close* each side’s nearest seed gets to the target operand (a normalised Hamming distance, 0 = exact hit). The non-VP arm stalls at a positive residual; the value_profile arm closes it.

Tool: `value_distance_reached`

Metric: `naive_min_distance` = the non-VP arm’s residual to the operand (> 0 = never lands it); `distance_gap` = how much further from the operand the loser stalls; `distance_closure_ratio` = fraction of that gap VP closes; `winner_closer` (bool)

Verify rule (the arbiter fires this, no LLM): `winner_closer == true AND distance_gap >= 0.15`

Real example: `lcms/2288 (cmsChannelsOfColorSpace, operand 'MCH2')` — the non-VP arm stalls at distance 0.75 (one of four bytes right); value_profile reaches 0.0, so `distance_gap`= 0.75 and `distance_closure_ratio`= 1.0 (closes the whole gap).



Without a gradient the loser cannot tell “one byte right” from “none right”; VP’s per-byte score gives it the slope to finish.

Sub-case: the operand sits at a FIXED offset (enrichment route). Five of these branches gate on a magic value at a *fixed* file offset — an ICC device-class signature, a FourCC, a size tag — so instead of a sliding-window distance we measure the same climb directly as byte *enrichment* at that offset: value_profile ratchets the corpus until the exact operand bytes pile up there (the end-state I2S reaches by pasting, arrived at by climbing). The mechanism is identical to the distance route above; only the measurement is anchored to the offset — exactly the dual measurement route `i2s_string_literal_substitution` uses (enrichment when the literal has a fixed offset, distance when it floats). These LWL/L.WL branches typically resolve under I2S too — the boundary where VP-pro touches I2S-pro — so the enrichment is a `cmplog-vs-naive` ratio and the VP attribution comes from the deciding pair.

Tool: `operand_enrichment`

Metric: `signed_target_enrich` = $\log_2 \frac{\text{corpus freq} + \epsilon}{\text{baseline freq} + \epsilon}$ at the gate offset (> 0 = the exact bytes piled up there); `decoy_enrich` (the competing value)

Verify rule (the arbiter fires this, no LLM): `signed_target_enrich` ≥ 1.0 AND `decoy_enrich` $<$ `signed_target_enrich`

Real example: `lcms/2143` (`validDeviceClass`, ICC device-class signature at offset `0x0C`) — `signed_target_enrich` = +17.85, `decoy_enrich` = -0.90: the exact class bytes pile up at the offset while the wrong classes drain. Same ICC-header gate family as the ‘MCH2’ example above, neighbouring offset — which is why splitting them by measurement tool was an artefact.

2. The gradient builds more structure: assembly depth `vp_gradient_assembly_depth` ($n = 20$)

Where category 1’s gradient closes the distance to one *value*, this category’s gradient produces more *structure* on the way to the gate. It has **two measurement routes** — the same two the joint family’s `joint_assembly_depth` bundles, here for a single technique: VP either drives the parser *deeper* per seed (route A, `depth_reach`) or admits a *larger / token-denser* valid corpus (route B, `struct_size_and_token_count`). Both are “more structure assembled”; which route fires is a property of the gate, not a different mechanism. (We earlier split these as two categories — *drives_assembly_depth* (15) and *admits_structurally_richer_corpus* (5) — but *size* alone cannot separate “drives *this* gate deeper” from “admits a bigger corpus in general” (and size is already route B’s metric), so they are one mechanism with two routes, merged 2026-06-26.)

Route A — drives the parser deeper (depth). The climb is *along a chain*: each loop iteration or nested field has its own comparison, and VP’s per-comparison reward keeps rewarding seeds that get one step further in, marching the parser *deeper* until it reaches the gate. The tell is not “the right bytes are present” — it is “the VP seeds execute the blocker line far more / drive it much further in.”

```
// libxml2 parser.c : xmlParseCharRef() (numeric char-ref &#NN;)
} else if ((RAW == '&') && (NXT(1) == '#')) {
    SKIP(2);
    while (RAW != ';') { // <-- blocker (libxml2/6466):
        digit-assembly loop
        if ((RAW >= '0') && (RAW <= '9')) // each pass consumes one
            more digit
            val = val * 10 + (RAW - '0');
        NEXT;
    }
}
```

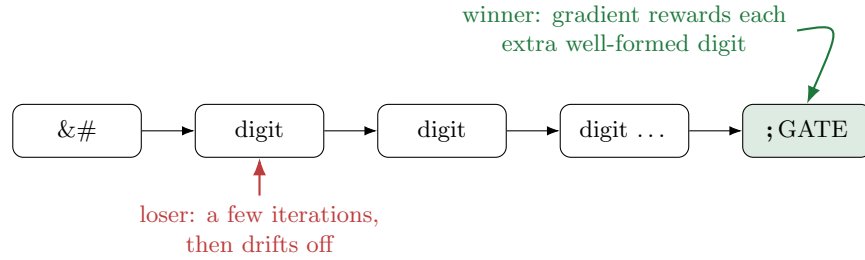
How we know. We run each side’s corpus and count, per seed, how many times the blocker line executes (its parse *depth*) and whether the seed takes the decisive side. The VP arm should reach the line far more.

Tool: `depth_reach`

Metric: `depth_lift` = winner / loser per-seed executions of the blocker line; `side_lift` = how much more the winner takes the decisive branch; `winner_deeper` (bool)

Verify rule (the arbiter fires this, no LLM): `depth_lift >= 1.5 AND winner_deeper == true`

Real example: libxml2/6466 (`xmlParseCharRef`, the `while (RAW != ';')` digit loop, `parser.c:2352`) — `depth_lift`= 9.07, `side_lift`= 7.8: the value-profile corpus drives the char-ref assembly loop $\approx 9\times$ deeper per seed than its non-VP partner.



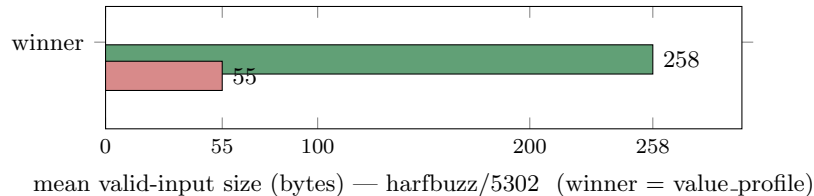
Each comparison on the chain emits its own partial-match reward, so VP’s corpus keeps walking deeper; the loser has no slope past the first steps.

Route B — admits a larger / token-denser corpus (size). The other route to “more structure” (the 5 branches formerly labelled *admits_structurally_richer_corpus*): extra valid structure (a longer well-formed body, more keywords/tokens) opens *more* comparison gradients downstream — so VP’s feedback rewards seeds that carry it, and the whole corpus drifts toward *bigger* or *more token-dense* valid inputs. The non-VP arm has no reason to grow its seeds, so it never accumulates the structure the gate needs. Two measurable sub-signals: more total valid *size*, or higher *token density*.

```
// a downstream gate only longer / token-dense valid inputs reach
parse_header(in); // every arm clears this
... build_body(in) ... // VP's corpus grows larger valid
    bodies here
if (have_enough_structure(in)) // <-- blocker: only the richer corpus
    arrives
    enter_deep_state(in);
```

How we know. We compare the winner’s and loser’s seed sets for this branch on two structural axes — mean valid *size* and *token density* — and require the VP side to be richer.

Tool: <code>struct_size_and_token_count</code>
Metric: <code>size_lift</code> = winner / loser mean valid size; <code>tag_lift</code> = relative count of structural tags; <code>token_density_lift</code> = winner / loser tokens-per-byte
Verify rule (the arbiter fires this, no LLM): <code>(size_lift >= 1.3 AND tag_lift >= 1.0) OR (token_density_lift >= 2.0 AND vp_token_density > naive_token_density)</code>
Real example: harfbuzz/5302 — <code>size_lift</code> = 4.69 (VP seeds average 258 B vs 55 B), <code>tag_lift</code> = 1: the VP corpus carries a much larger valid font body. Token-density sub-case sqlite3/14588 — <code>token_density_lift</code> = 34.6 (0.154 vs 0.004 tokens/byte): VP’s corpus is far more SQL-token-dense.



VP rewards the extra structure that opens further gradients, so its corpus grows the valid body (`size_lift` = 4.69); the loser stays small.

3. Incremental conjunction climb `vp_incremental_conjunction_climb` (a *recovered* category)

The category the automated pipeline *missed*. The gate is not one multi-byte compare but a *conjunction* of ≥ 2 narrow equality checks against constants that together form a contiguous magic / signature / byte-order mark (BOM): `if (b[0]==C0 && b[1]==C1 && ...)`. `value_profile` climbs it *one clause at a time* — each `&&` term emits its own `CMP_MAP` partial-match reward *and* its own short-circuit coverage edge — so its corpus reaches a deeper prefix of the sequence than the loser, which substitutes/guesses one operand at a time and stalls partway. This is exactly the paper’s motivating example (Fig. 2a):

```
// libxml2 encoding.c : xmlDetectCharEncoding() -- recognise a UCS-4
// BOM
if ((in[0] == 0x00) && (in[1] == 0x00) &&
    (in[2] == 0x00) && (in[3] == 0x3C)) // <-- blocker (libxml2
    /6337)
    return(XML_CHAR_ENCODING_UCS4BE); // value_profile
    resolves 9/10; // naive blocks 6/10
```

Why it is a FALSE NEGATIVE. Two compounding reasons kept it out of the discovered taxonomy:

- **Admission filter (primary).** A branch enters the dataset only under the *decisive* rule — winner resolves $\geq 8/10$ *and* loser blocks $\geq 8/10$. A short-circuit `&&` conjunction opens a *new coverage edge for every clause it passes*, so even the loser arm gets a partial *coverage* foothold and resolves the gate often enough that `loser_blocked` < 8. All-or-nothing blocking is the wrong test for this gate shape: in `libxml2`’s `xmlDetectCharEncoding`, **all 10** BOM gates score **0/10** under the strict rule and never reach the LLM classifier at all — including Fig. 2a.

- **Distiller collapse (secondary).** The two keyword conjunctions that *did* squeak through ((RAW=='I') && (NXT(1)=='D') → "ID", libxml2/6621; ... 'A'&&'N'&&'Y' → "ANY", libxml2/6663) were normalised by Pass A into a single `multibyte_literal` operand — indistinguishable from a one-shot `memcmp` — so the LLM folded them into category 1.

How we recover it (two new tools, no LLM). A *static* identifier separates this shape from category 1, and a *dynamic* arbiter confirms the climb. Both read data we already have.

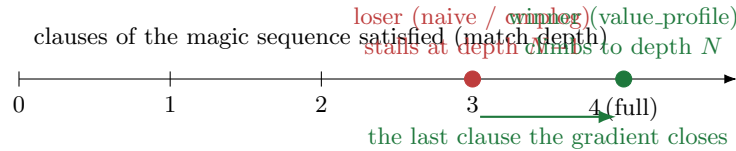
Tool: `detect_conjunction_gate`
Metric: STATIC. From the `llvm-cov` branch report, the gate line carries ≥ 2 short-circuit branch-points (**Branch** (952:6), (952:25), ... — clause $k+1$'s executions = clause k 's **True** count) AND ≥ 2 per-byte == clauses at *consecutive* input positions, no range / ||. Emits `is_magic_sequence`, `n_clauses`. G2-clean: it never looks at who resolved.
Verify rule (the arbiter fires this, no LLM): `is_magic_sequence == true AND n_clauses >= 2`
Real example: libxml2/6337 (Fig. 2a) — `n_clauses=4`, staircase confirmed; cleanly distinct from a single-compare cat-1 gate (one branch-point).

Tool: `conjunction_match_depth`
Metric: DYNAMIC. Per arm, the deepest prefix of the magic sequence its corpus reached (0..N). The `value_profile` winner climbs deeper; the loser stalls. `match_depth_gap = winner_match_depth - loser_match_depth`.
Verify rule (the arbiter fires this, no LLM): `n_clauses >= 2 AND winner_deeper AND winner_match_depth >= n_clauses-1 AND match_depth_gap >= 1`
Real example: libxml2/6663 ("ANY") — `value_profile` reaches depth 2 ("AN"), `cmplog` stalls at depth 1 ("A"); libxml2/6621 ("ID") — `vp` depth 2 vs `cmplog` 1. The loser literally lands all-but-one byte, never the whole sequence.

Relaxed admission, and how many it retrieves. For a statically-confirmed conjunction gate we lower *only* the loser-side bar from `loser_blocked` ≥ 8 to ≥ 4 (keeping `winner_resolved` ≥ 8), because the `&&` coverage leak structurally depresses `loser_blocked`; the climb is then confirmed by the match-depth arbiter above (the real discriminator). Over libxml2's `xmlDetectCharEncoding`:

admission	gates retrieved	which
strict (≥ 8 / ≥ 8)	0	— (entire family filtered out)
relaxed (≥ 8 / ≥ 4)	5	UCS4BE (Fig. 2a), UCS4-2143, UCS4-3412, UTF16LE, UTF16BE
relaxed (≥ 8 / ≥ 3)	6	+ UCS4LE
not blockers (all-resolve / weak)	4	2-byte BOMs, EBCDIC, UTF8-BOM

So in this one function we recover **5** BOM blockers (incl. the Fig. 2a gate) by relaxation, plus the **2** keyword conjunctions ("ID", "ANY") reclaimed from category 1 — **7** members, where the strict pipeline found *zero*. A cross-target census (PNG/GIF/file magic, other signature gates) is pending and expected to add more.



The conjunction climb: each `&&` clause is a step. The loser gets within one byte (so it leaks coverage and dodges the $\geq 8/8$ filter), but only `value_profile`'s per-clause gradient takes the final step.

At a glance

Category	<i>n</i>	Tool / metric	One-line reason
operand climb	33	value_distance_reached (floating) / operand_enrichment (fixed offset)	climb to the exact gate — by distance, or offset enrichment
assembly depth	20	depth_reach (depth route) / struct_size_and_token_count (size route)	the gradient builds gate structure — drives the gate per seed (depth), a larger / token-denser (size)
VP-pro total (auto-discovered) 53			
incremental conjunction climb †	7*	detect_conjunction_gate + conjunction_match_depth	climb a multi-byte BOM sequence clause

† *Recovered* category (§3): a false negative of the auto-pipeline — the strict $\geq 8/8$ admission filters out every short-circuit-~~ed~~ conjunction gate (the clause coverage leak depresses `loser_blocked`), so it never reached the classifier. * 2 keyword conjunctions reclaimed from category 1 ("ID", "ANY") + 5 BOM gates retrieved from `libxml2 xmlDetectCharEncoding` by relaxed admission ($\geq 8 / \geq 4$) and confirmed by the match-depth arbiter; the 5 are *not* part of the 53 auto-discovered total, and a cross-target census is pending.

Counts and example metrics regenerated from `bench/dataset.jsonl` (cats 1–2) and the conjunction tools (cat 3); verify rules are the deterministic arbiter rules (no LLM in the decision). `lcms/libxml2` gates are verified against local source dumps; `harfbuzz/sqlite3` gates are from their analysis cards (their corpora live on the other server). The common thread across all categories is the *partial-match gradient*: `value_profile` turns a wrong/right cliff into a slope, which it then climbs — to the exact operand (1, by sliding-window distance for a floating operand or by fixed-offset enrichment), by building more structure (2, driving the gate deeper per seed or admitting a larger / token-denser corpus), or up a multi-byte conjunction one clause at a time (3, the recovered category).